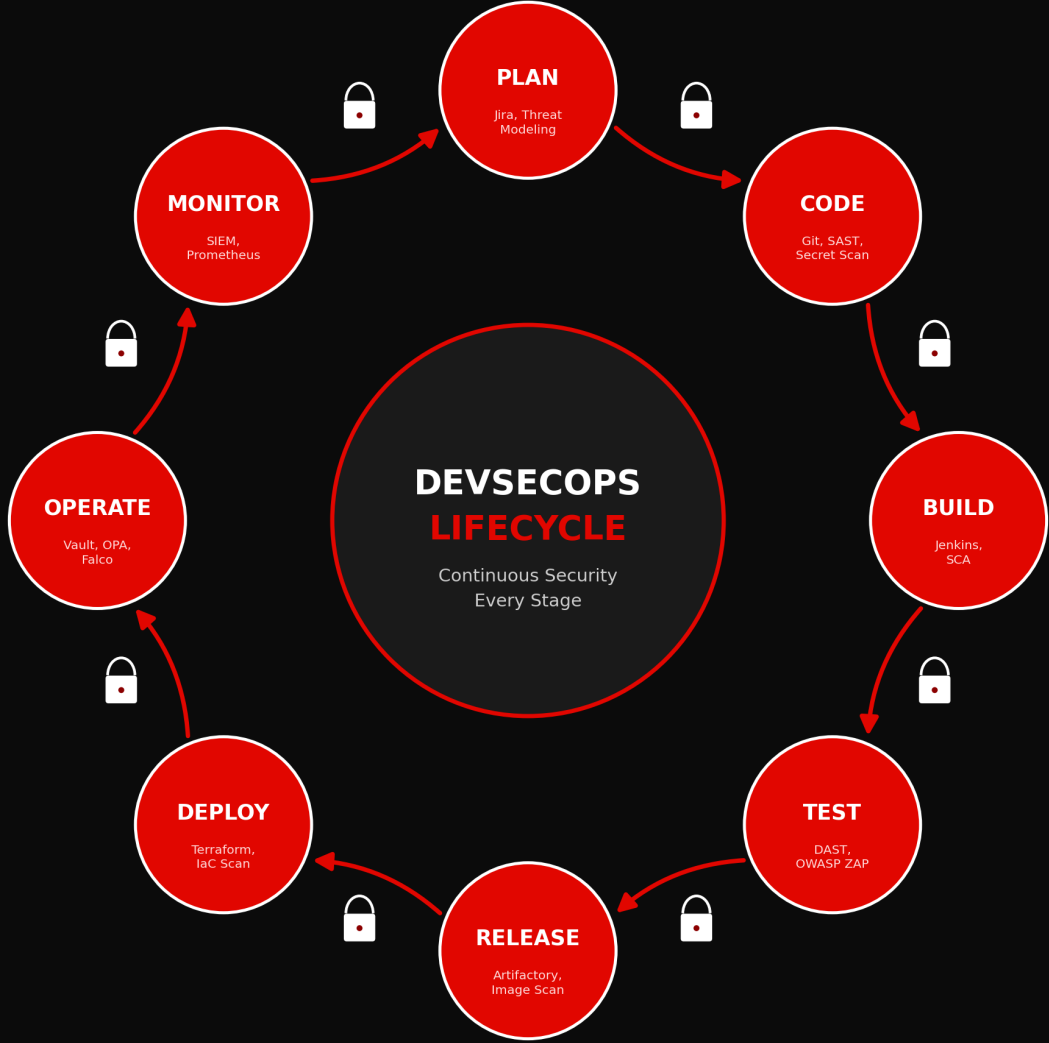


DEVSECOPS

FULL LIFECYCLE WORKFLOW

Detailed Edition — Diagrams, Toolchain, Gates & Rationale for Every Stage

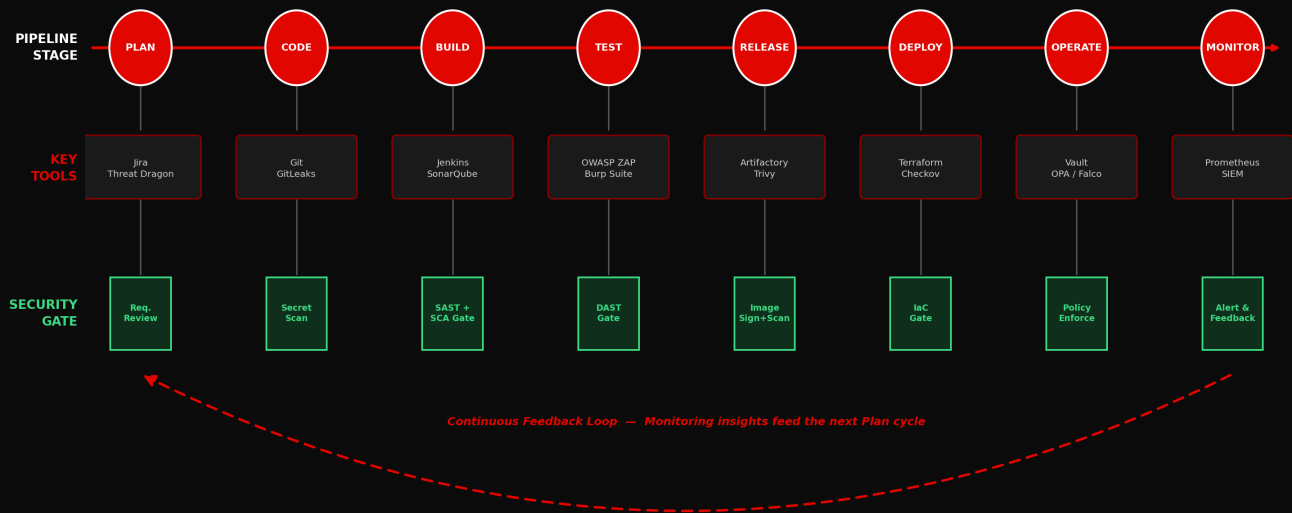


CONTENTS

1. Lifecycle Overview Diagram — the continuous security loop
2. End-to-End Pipeline Architecture — tools & gates, swimlane view
3. Shift-Left Cost Curve — why earlier detection wins
- 4–11. Stage-by-Stage Deep Dive — Plan, Code, Build, Test, Release, Deploy, Operate & Secure, Monitor — each with its own
Input → Tools → Gate → Output flow diagram

END-TO-END PIPELINE ARCHITECTURE

Stage → Tooling → Security Gate, mapped across the full pipeline



WHY SHIFT LEFT

The economic case for embedding security early in the lifecycle

COST TO FIX A VULNERABILITY BY STAGE FOUND



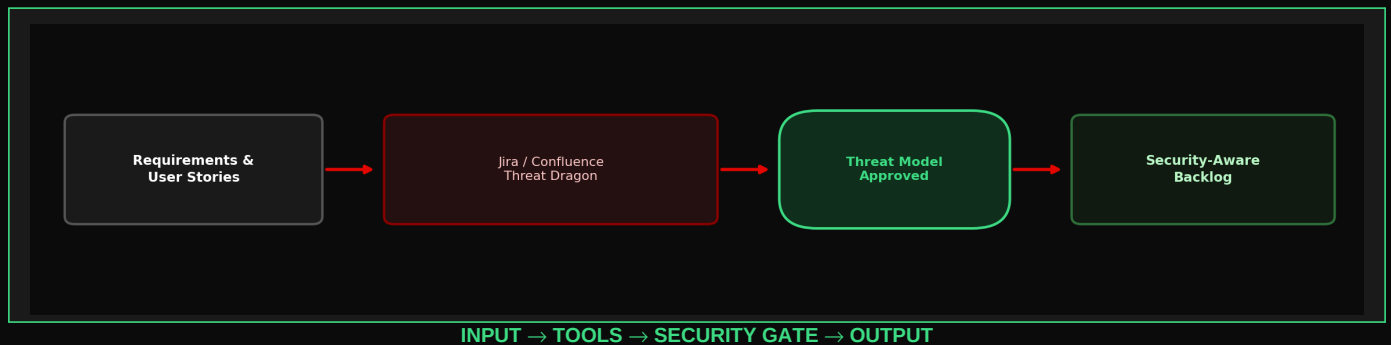
Relative cost multiplier (illustrative) — the earlier a flaw is caught, the cheaper it is to fix. This is the core argument for “shifting security left” across the DevSecOps lifecycle.

STAGE-BY-STAGE DEEP DIVE

Each stage below: what happens, the flow diagram, key tools, and why it matters

01 PLAN

Define security requirements, threat models, and compliance criteria before a line of code is written.



Security & dev teams collaborate on user stories with abuse-case scenarios. Threat modeling (STRIDE/DREAD) identifies attack surfaces early. Compliance requirements (PCI-DSS, HIPAA, SOC2) are mapped to acceptance criteria before sprint planning locks in scope.

KEY TOOLS

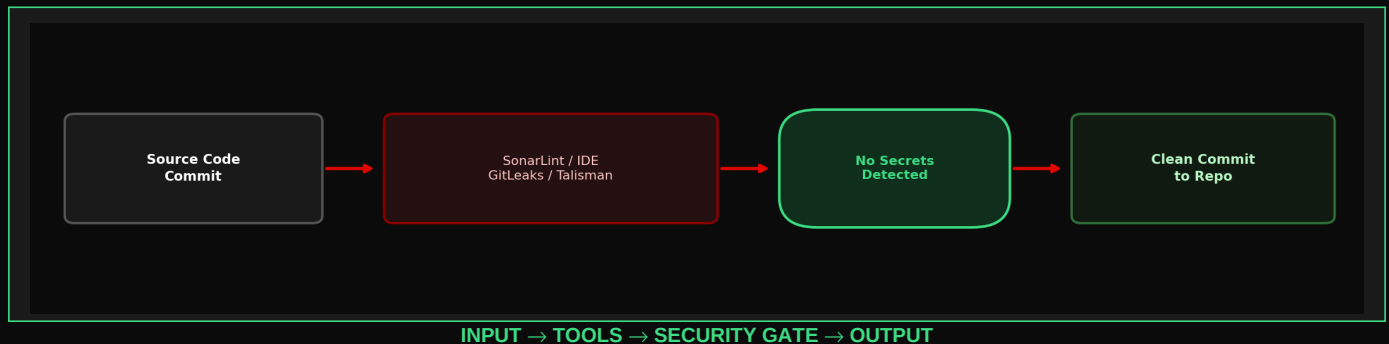
Jira, Confluence, OWASP Threat Dragon, Microsoft Threat Modeling Tool, IriusRisk

WHY IT MATTERS

Vulnerabilities caught here cost ~10-100x less to fix than in production. A missed requirement is the cheapest bug you'll ever find.

02 CODE

Developers write code with security guardrails active in the IDE and at commit time.



IDE plugins flag insecure patterns in real time (SQLi, XSS, hardcoded secrets) as the developer types. Pre-commit hooks block secrets and run linting before code reaches the shared repo. Peer code review includes a security checklist alongside functional review.

KEY TOOLS

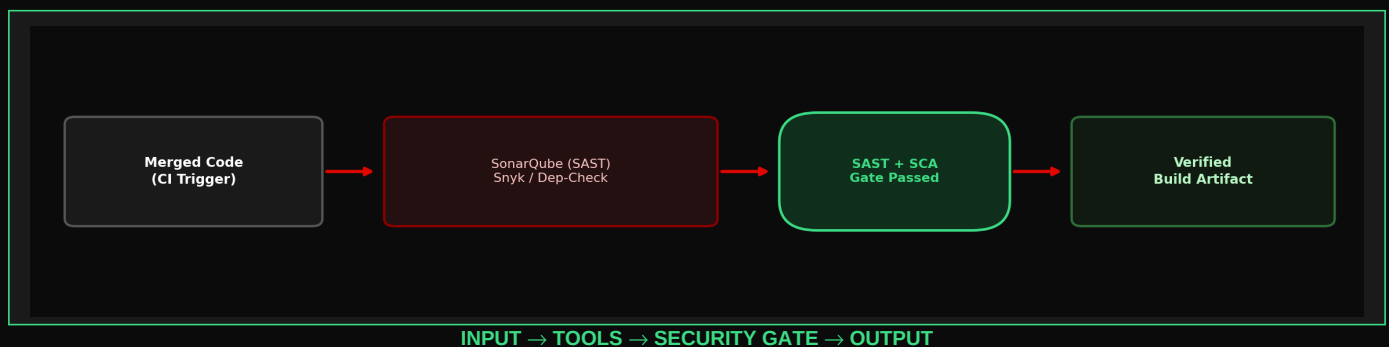
Git, SonarLint, GitLeaks, TruffleHog, pre-commit framework, Talisman

WHY IT MATTERS

Shifting scanning left of the commit prevents secrets and injection flaws from ever entering version history — where they're expensive to purge.

03 BUILD

CI pipeline compiles code and runs automated static & composition analysis on every commit.



SAST scans source code for vulnerable patterns (injection, insecure deserialization, broken auth logic). SCA checks all third-party and open-source dependencies against CVE databases for known vulnerabilities and license risk. The build fails automatically if critical issues are found — a hard quality gate.

KEY TOOLS

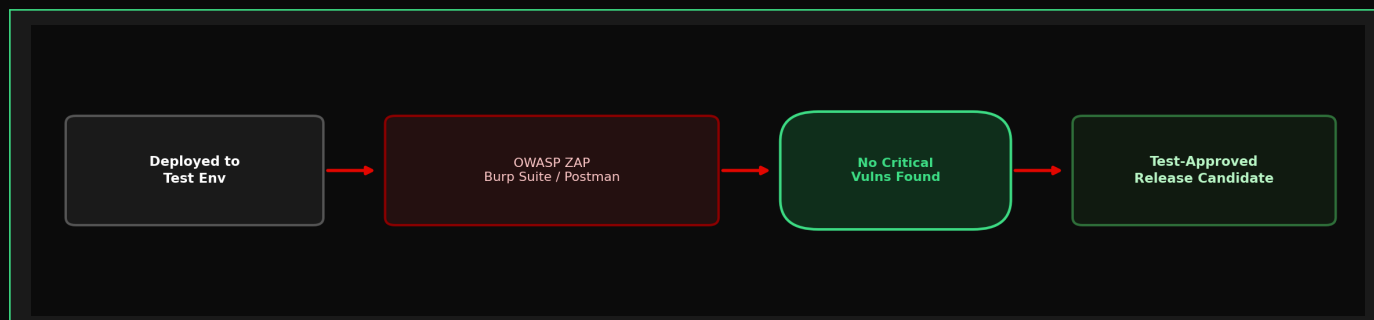
Jenkins, GitHub Actions, GitLab CI, SonarQube, Semgrep, Snyk, OWASP Dependency-Check

WHY IT MATTERS

A broken build on a critical CVE is inconvenient. A breach caused by a known CVE that shipped anyway is a headline.

04 TEST

Application is deployed to a test environment and attacked in a controlled way.



INPUT → TOOLS → SECURITY GATE → OUTPUT

DAST tools crawl and attack the running app (SQLi, XSS, auth bypass, broken access control) the way a real attacker would, with no knowledge of source code. API security tests validate authentication, authorization, rate limiting, and input handling. IAST combines code instrumentation with runtime traffic analysis for deeper coverage than SAST or DAST alone.

KEY TOOLS

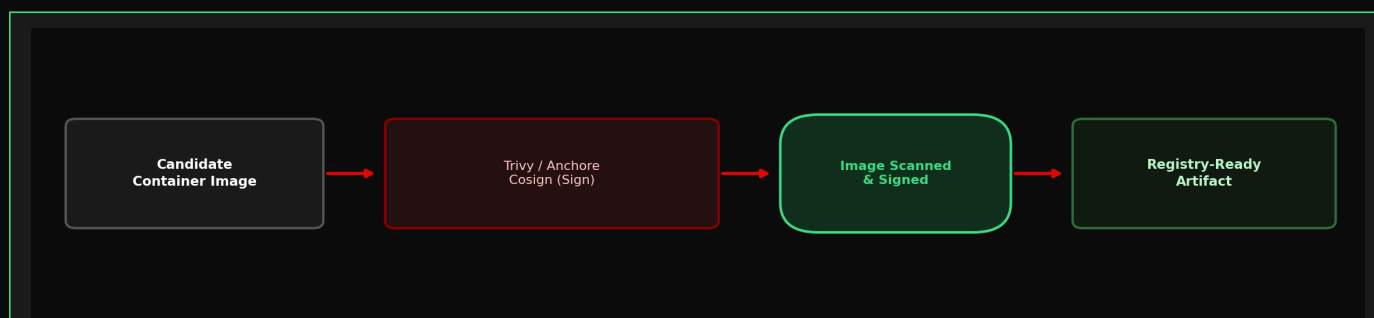
OWASP ZAP, Burp Suite, Postman (security test collections), Contrast Security

WHY IT MATTERS

Testing the running application catches issues static analysis alone cannot see — business logic flaws, broken auth, and chained exploits.

05 RELEASE / PACKAGE

Approved builds are packaged as signed, scanned artifacts ready for deployment.



INPUT → TOOLS → SECURITY GATE → OUTPUT

Container images are scanned for OS and library vulnerabilities before being pushed to a registry. Artifacts are cryptographically signed to guarantee provenance and prevent tampering in transit. Only signed, scanned images are allowed to progress to the deploy stage — unsigned images are rejected by policy.

KEY TOOLS

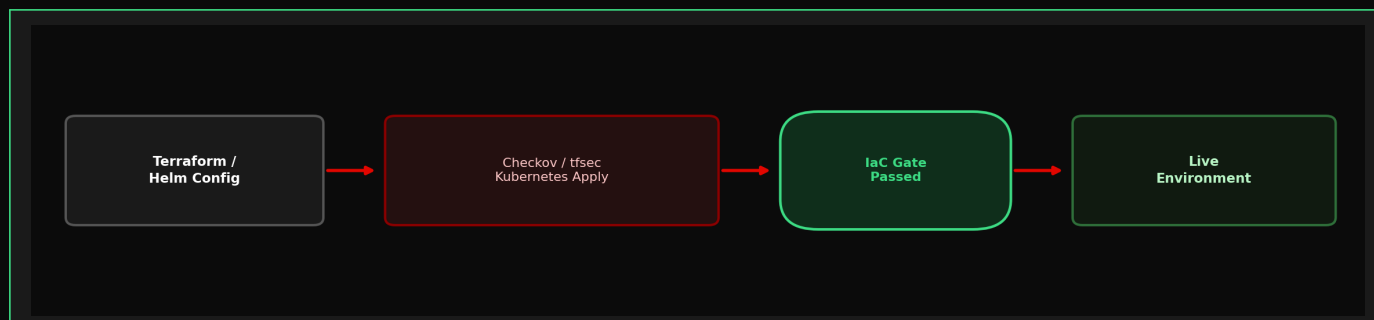
TFrog Artifactory, Sonatype Nexus, Trivy, Anchore, Clair, Cosign (Sigstore), Notary

WHY IT MATTERS

Signing and scanning here stops a compromised or vulnerable image from ever reaching a production cluster — this is the last gate before infrastructure.

06 DEPLOY

Infrastructure and application are provisioned and released using version-controlled, scanned IaC.



INPUT → TOOLS → SECURITY GATE → OUTPUT

Infrastructure as Code (Terraform, CloudFormation, ARM) defines environments in a repeatable, auditable, peer-reviewed way. IaC is statically scanned for misconfigurations — open security groups, public storage buckets, overly permissive IAM — before every apply. Deployments roll out via blue-green or canary strategy to limit blast radius if something slips through.

KEY TOOLS

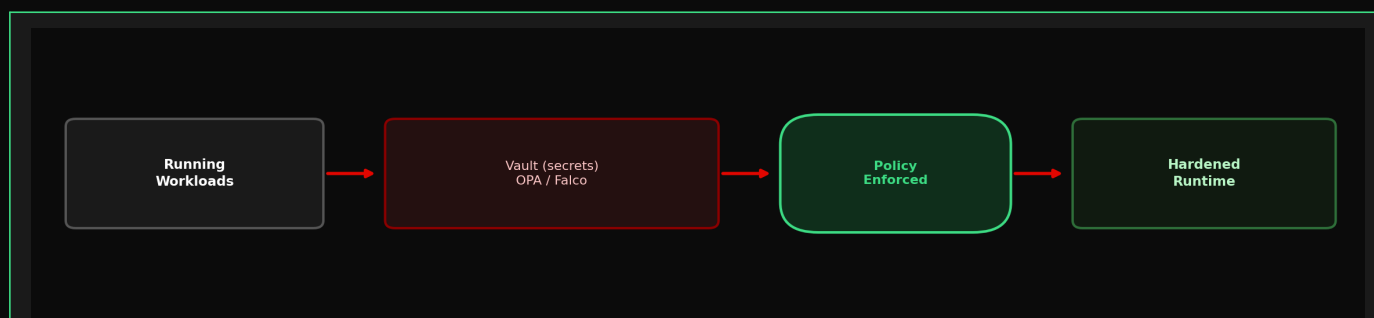
Terraform, Ansible, Checkov, tfsec, Terrascan, Kubernetes, Helm

WHY IT MATTERS

Misconfigured cloud infrastructure is one of the single largest causes of real-world breaches — catching it pre-apply, not post-incident, is the whole point of this gate.

07 OPERATE & SECURE

Running systems are hardened, secrets are centrally managed, and policy is enforced continuously.



INPUT → TOOLS → SECURITY GATE → OUTPUT

Secrets (API keys, DB credentials, tokens) are stored in a vault and injected at runtime — never committed to code or config files. Policy-as-code enforces guardrails automatically across the fleet (e.g. 'no privileged containers', 'no public load balancers'). Runtime security tools continuously watch for anomalous process, file, and network behavior inside containers and hosts.

KEY TOOLS

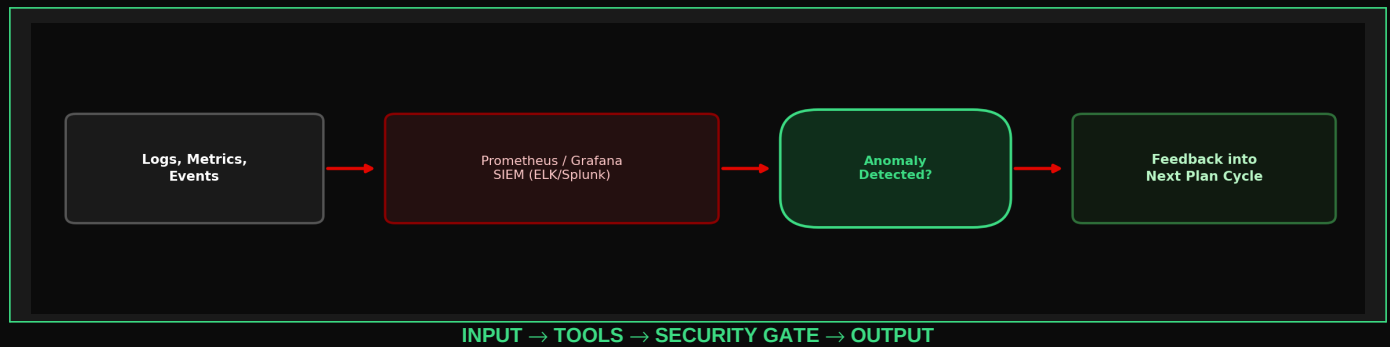
HashiCorp Vault, AWS Secrets Manager, Open Policy Agent, Falco, Kube-bench, Prisma Cloud

WHY IT MATTERS

Runtime protection is the safety net for the unknown unknowns — the zero-days and novel attack chains that slip past every earlier gate.

08 MONITOR

Logs, metrics, and alerts feed continuous visibility and drive the next Plan cycle.



Centralized logging (SIEM) correlates events across the whole stack — app, infra, network, identity — to detect attacks that look innocuous stage-by-stage. Metrics dashboards track system health and security KPIs like Mean Time to Detect and Mean Time to Respond. Incidents trigger automated alerting and response playbooks, and post-incident lessons feed directly back into the next Plan cycle.

KEY TOOLS

Prometheus, Grafana, ELK Stack, Splunk, Wazuh, PagerDuty, TheHive

WHY IT MATTERS

This is what makes it a cycle, not a line: monitoring insights become next sprint's threat model and security requirements.